

Unity & C#

Ejercicios prácticos — Métodos y clases

Contenido

Ejercicio 1 — El cofre del tesoro

Ejercicio 2 — Sistema de salud

Ejercicio 3 — Daño de proyectiles

Ejercicio 4 — Inventario del personaje

Ejercicio 5 — Sistema de puntuación

Ejercicio 1 — El cofre del tesoro

CONTEXTO DE JUEGO

Tu personaje encuentra un cofre en una mazmorra. Al acercarse, el juego debe registrar en consola el tipo de cofre, el nombre del héroe y un mensaje de bienvenida. Toda la información por Debug.Log.

PROBLEMA A RESOLVER

¿El método necesita devolver un valor para ser útil? ¿Necesita datos externos o puede operar solo con variables globales de la clase?

CONSIGNA TÉCNICA

- Crear un método **AbrirCofre()** de tipo void
- Recibir un parámetro string nombre Heroe
- Mostrar en consola: nombre del héroe + tipo de cofre + recompensa
- Llamarlo desde Start()

DESAFÍO DE RAZONAMIENTO

Si cambias el orden del parámetro y agregas un segundo parámetro int monedas, ¿que ocurre si los llamas en orden invertido al invocar el método? ¿Que error genera Unity y por que?

PISTAS

- Un método void no usa la palabra return — solo ejecuta acciones.
- Las variables declaradas fuera de los métodos son accesibles desde cualquier método del script.
- Podes concatenar en Debug.Log usando + o interpolación con \$"texto {variable}".
- Verifica que el tipo del parámetro en la firma coincida con el argumento al llamarlo.

Ejercicio 2 — Sistema de salud

CONTEXTO DE JUEGO

Tu personaje tiene puntos de vida (HP). Al recibir daño o curación, el sistema debe actualizar la salud y reportarlo en consola. Un enemigo débil quita 10 HP, un jefe quita 50 HP y una poción restaura 30 HP.

PROBLEMA A RESOLVER

¿Conviene un único método que reciba el tipo de acción y el valor, o métodos separados? ¿Qué parámetros son int y cuáles podrían ser bool?

CONSIGNA TÉCNICA

- Método `ModificarSalud(int cantidad, bool esDaño)`
- Si `esDaño` es `true`, restar; si es `false`, sumar
- Variable global `int vidaActual` inicializada en 100
- Mostrar en consola: estado anterior, acción y vida resultante
- Llamarlo 3 veces desde `Start()` con distintos valores

ESTRUCTURA ESPERADA EN CONSOLA

```
HP antes: 100 | Daño recibido: 50 | HP ahora: 50
HP antes: 50  | Curación: 30      | HP ahora: 80
HP antes: 80  | Daño recibido: 10    | HP ahora: 70
```

DESAFÍO DE RAZONAMIENTO

Si invirtieras el orden de los parámetros y llamarás `ModificarSalud(true, 50)` en lugar de `ModificarSalud(50, true)`, ¿Unity lo acepta sin errores? ¿El error aparece en compilación o en ejecución?

PISTAS

- El operador ternario condicional `? valor True : valor False` puede simplificar el condicional.
- Para evitar que la vida baje de 0, explora `Math.Max(0, valor)`.
- Las variables globales en un `MonoBehaviour` se declaran al inicio de la clase.
- C# es fuertemente tipado: pasar un `bool` donde se espera un `int` es error de compilación.

Ejercicio 3 — Daño de proyectiles

CONTEXTO DE JUEGO

Un sistema de combate calcula el daño final de un proyectil considerando el daño base, un multiplicador de crítico y la resistencia del enemigo. El resultado calculado por un método es usado por otro para aplicar el efecto.

CONSIGNA TÉCNICA

- Metodo `int CalcularDano(int danoBase, float multiplicador, int resistencia)`
- Fórmula: $(\text{danoBase} * \text{multiplicador}) - \text{resistencia}$, mínimo 1
- Método `void AplicarDano(int danoFinal, string nombreEnemigo)`
- Llamar a `AplicarDano` pasandole el retorno de `CalcularDano` como argumento
- Mostrar nombre del enemigo, daño final y si fue golpe crítico ($\text{multiplicador} > 1.5f$)

FLUJO ESPERADO DEL CÓDIGO

```
int resultado = CalcularDano(30, 2.0f, 5);
AplicarDano(resultado, "Orco Jefe");

// Consola:
// [CRITICO] Orco Jefe recibe 55 puntos de dano.
```

DESAFÍO DE RAZONAMIENTO

¿Qué sucede si eliminas el `return` del método `Calcular Daño` pero mantienes el tipo `int` en la firma? ¿El error aparece al compilar o al ejecutar? Además: ¿podría este método declararse como `void` si solo se necesita mostrar el resultado en consola? Explica cuándo conviene cada enfoque.

PISTAS

- El tipo antes del nombre del método (`int`, `float`, `string`) indica que tipo debe devolver el `return`.
- Puedes usar el retorno directamente como argumento: `AplicarDano(CalcularDano(30, 2f, 5), "Orco")`.
- Para el mínimo de daño 1, investiga `Mathf.Max(1, valor)` o un `if`.
- Si un método declara retorno `int` y no tiene `return`, C# general error: 'not all code paths return a value'.

Ejercicio 4 — Inventario del personaje

CONTEXTO DE JUEGO

Tu personaje puede recoger objetos del mundo. El inventario tiene un límite de peso: si el objeto a recoger supera el espacio disponible, la acción falla y debe notificarse. Un método debe verificar si hay espacio y devolver el resultado para que otro método decida si ejecuta la recogida o muestra un aviso de inventario lleno.

PROBLEMA A RESOLVER

¿Por qué el método de verificación debe retornar bool en lugar de void? ¿Cómo usas ese valor de retorno para controlar el flujo en otro método?

CONSIGNA TÉCNICA

- Variable global float pesoActual = 0f y float pesoMaximo = 20f
- Método bool HayEspacio(float peso Objeto): retorna true si cabe, false si no
- Método void RecogerObjeto(string nombre, float peso): llama a HayEspacio internamente
- Si HayEspacio retorna true: sumar el peso y mostrar 'Recogiste [nombre]. Peso actual: X/20'
- Si retorna false: mostrar 'No puedes cargar [nombre]. Inventario lleno.'
- Probar desde Start() con al menos 4 objetos de distintos pesos

EJEMPLO DE SALIDA EN CONSOLA

```
Recogiste Espada. Peso actual: 8.5/20
Recogiste Escudo. Peso actual: 15/20
No puedes cargar Armadura Pesada. Inventario lleno.
Recogiste Poción. Peso actual: 17/20
```

DESAFÍO DE RAZONAMIENTO

¿Que diferencia hay entre escribir `if (HayEspacio(peso) == true)` y simplemente `if (HayEspacio(peso))`? ¿Son iguales? Además: si pesoActual fuera una variable local dentro de RecogerObjeto en lugar de global, ¿que problema surgió al llamar el método varias veces?

PISTAS

- bool es un tipo de retorno válido: `return true;` o `return false;` son sentencias completas.
- Podes usar la llamada al método directamente en el if: `if (HayEspacio(peso)) { ... }`

- Las variables globales (de clase) mantienen su valor entre llamadas; las locales se resetean.
- Usa `pesoActual += peso` para acumular; no reemplaces el valor sino que lo sumes.
- `Mathf.Round(valor * 10f) / 10f` te ayuda a mostrar decimales limpios en consola.

Ejercicio 5 — Sistema de puntuación

CONTEXTO DE JUEGO

Al finalizar un nivel, el juego calcula la puntuación del jugador basándose en enemigos eliminados, tiempo restante y un bonus por dificultad. Luego convierte ese número en una calificación de letra (S, A, B, C, D) y muestra un resumen completo en consola.

PROBLEMA A RESOLVER

¿Cómo organizar el flujo para que cada método tenga una sola responsabilidad? ¿Cual método calcula, cual clasifica y cual muestra? ¿Qué datos necesita recibir cada uno y que debe retornar?

CONSIGNA TÉCNICA

- `int CalcularPuntaje(int enemigos, int tiempoRestante, float bonusDificultad)`
- Fórmula: `Mathf.RoundToInt((enemigos * 100 + tiempoRestante * 10) * bonusDificultad)`;
- `string ObtenerCalificacion(int puntaje):` retorna 'S', 'A', 'B', 'C' o 'D'
- `S >= 5000 · A >= 3500 · B >= 2000 · C >= 1000 · D < 1000`
- `void MostrarResumen(string nombreJugador, int puntaje, string calificación)`
- Imprime el resumen final con todos los datos
- Desde `Start()`: encadenar las 3 llamadas pasando el retorno de una a la siguiente
- Probar con al menos 3 combinaciones distintas de parámetros

EJEMPLO DE SALIDA EN CONSOLA

```
===== RESULTADO FINAL =====  
Jugador : NewPlayer  
Enemigos: 35 | Tiempo: 45s | Dificultad: x1.5  
Puntaje : 5925  
Rango   : S  
=====
```

DESAFÍO DE RAZONAMIENTO

Actualmente `CalcularPuntaje` retorna `int`. Si el bonus de dificultad genera decimales (ej. 5925.75), ¿que se pierde al usar `int`? ¿Conviene cambiar el retorno a `float` o a `Mathf.RoundToInt()`? Justifica tu decisión y explica cómo afecta al método `ObtenerCalificacion`.

PISTAS

- Guarda el resultado de `CalcularPuntaje` en una variable local antes de pasarlo a `ObtenerCalificacion`.
- Usa `if / else if` encadenados para los rangos: va de mayor a menor (primero S, al final D).
- Un método que retorna `string` necesita `return "S"`; en cada rama — C# exige que TODOS los caminos retornen.
- Puedes llamar `MostrarResumen(nombre, CalcularPuntaje(...), ObtenerCalificacion(...))` en una sola línea.
- Para el formato de consola, `string.Format()` o la interpolación `$""` con alineación :N0 ayudan a organizar columnas.